COMP4616 – Fundamentals of Machine Learning

ASST. PROF. TUĞBA ERKOÇ

SPRING 2026

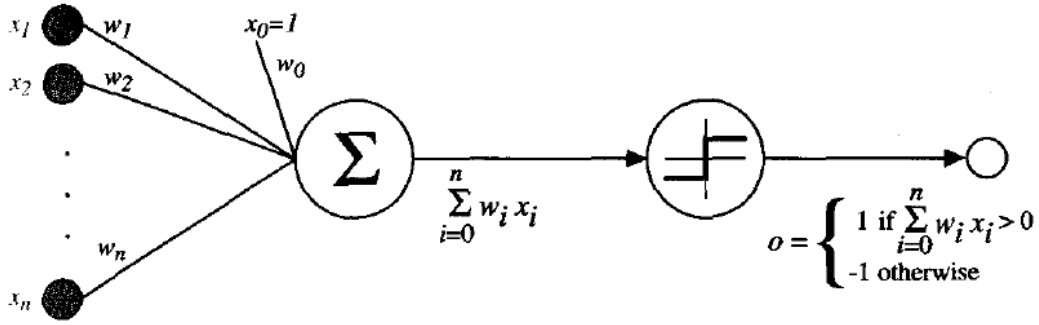
FROM PERCEPTRON TO MULTILAYER NEURAL NETWORKS

Neural Networks

- A neural network is a machine learning model that learns patterns from examples
- It receives input values, combines them using weights, and produces an output prediction
- During training, the network compares its prediction with the correct answer and slowly changes its weights to make better predictions next time

Input	Task	Output
House size, number of rooms, location	Predict house price	Price
Email text	Detect spam	Spam / not spam
Image pixels	Recognize object	Cat / dog / car

Perceptron



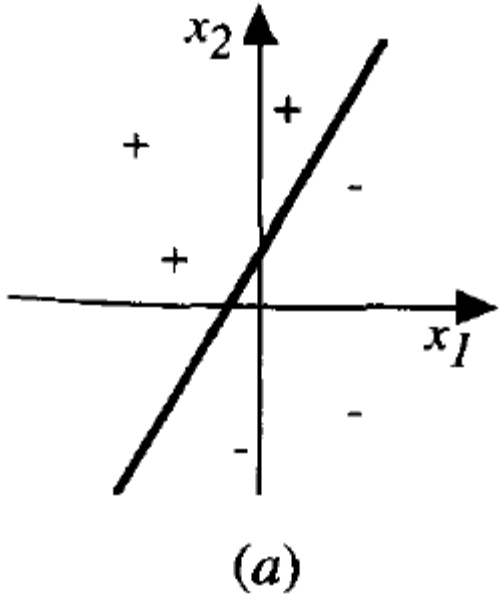
- In the late 1950s, psychologist and computer scientist Frank Rosenblatt developed the perceptron, a mathematical model inspired by the human brain's neurons.
- A **perceptron** takes a vector of real valued inputs, calculates a linear combination of these inputs, then outputs a 1 if the result is greater than some threshold and -1 otherwise

$$o(x_1, x_2, \dots, x_n) = \begin{cases} 1 & if \quad w_0 + w_1 x_1 + \cdots + w_n x_n > 0 \\ -1 & otherwise \end{cases}$$

- Learning a perceptron involves choosing values for the weights $w_0, \dots, w_n$.
- Therefore, the space **H** of candidate hypotheses considered in perceptron learning is the set of all possible real valued weight vectors.

Perceptron

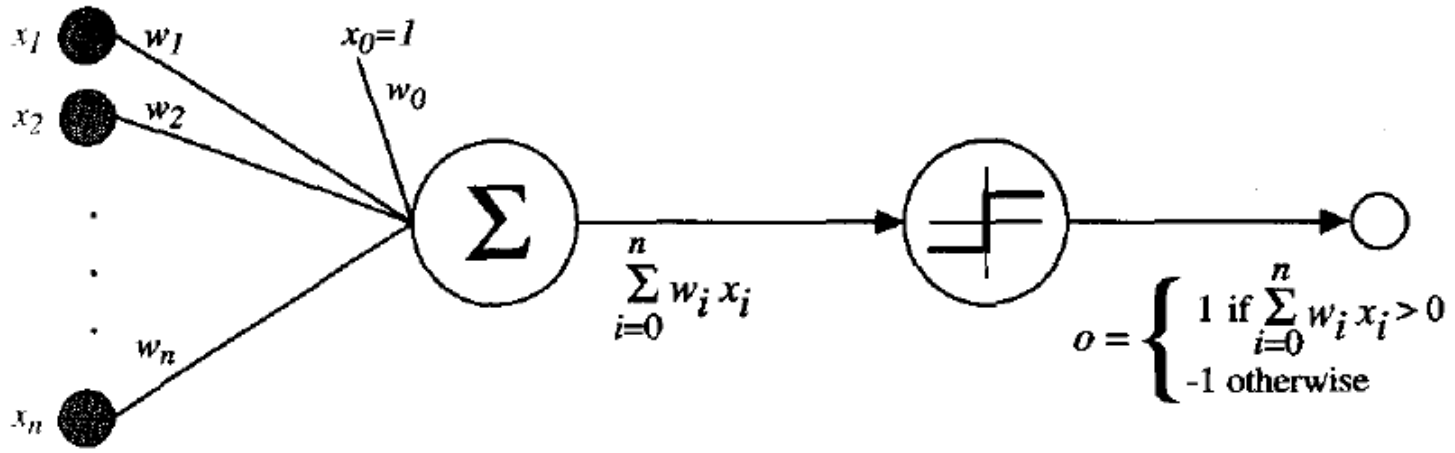
- We can view the perceptron as representing a hyperplane decision surface in the n-dimensional space of instances (i.e., points).
- The perceptron outputs a 1 for instances lying on one side of the hyperplane and outputs a -1 for instances lying on the other side.
- A single perceptron can only draw a linear boundary



(a)

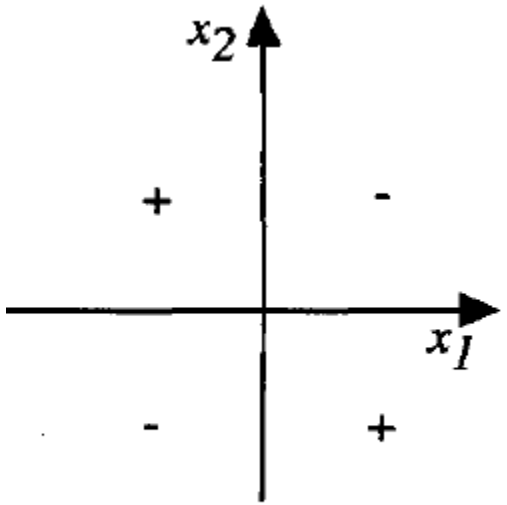
Perceptron

- Perceptron can represent all of the primitive boolean functions AND, OR, NAND (¬AND), and NOR (¬OR).
- For example, if we assume boolean values of 1 (true) and -1 (false), then one way to use a two-input perceptron to implement the AND function is to set the weights w_0= -0.8, and $w_1 = w_2 = 0.5$.



Perceptron

- Some Boolean functions cannot be represented by a single perceptron, such as the XOR function whose value is 1 if and only if $x_1 \neq x_2$.
- Note the set of linearly non separable training examples shown in below figure corresponds to this XOR function.
- See this demo.



(b)

Learning as Error Reduction

- A neural network does not know the correct weights at the beginning
- It starts with random weights, makes predictions, and checks how wrong those predictions are.
- This error is measured using a **loss function**
- The goal of training is to change the weights step by step so that the loss becomes smaller
- Gradient descent tells the model how to change each weight to reduce the error

$$w \leftarrow w - \eta \frac{\partial E}{\partial w}$$

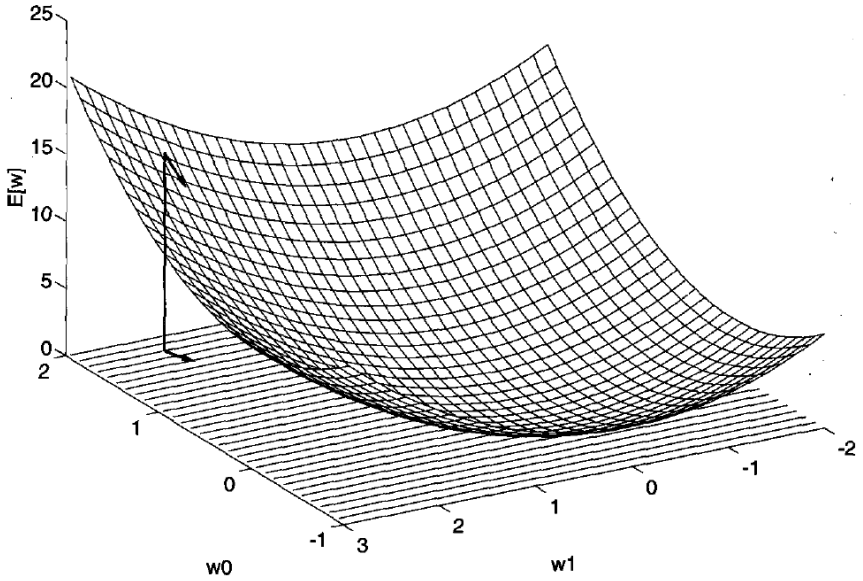


FIGURE 4.4
Error of different hypotheses. For a linear unit with two weights, the hypothesis space H is the w_0, w_1 plane. The vertical axis indicates the error of the corresponding weight vector hypothesis, relative to a fixed set of training examples. The arrow shows the negated gradient at one particular point, indicating the direction in the w_0, w_1 plane producing steepest descent along the error surface.

Gradient Descent Algorithm

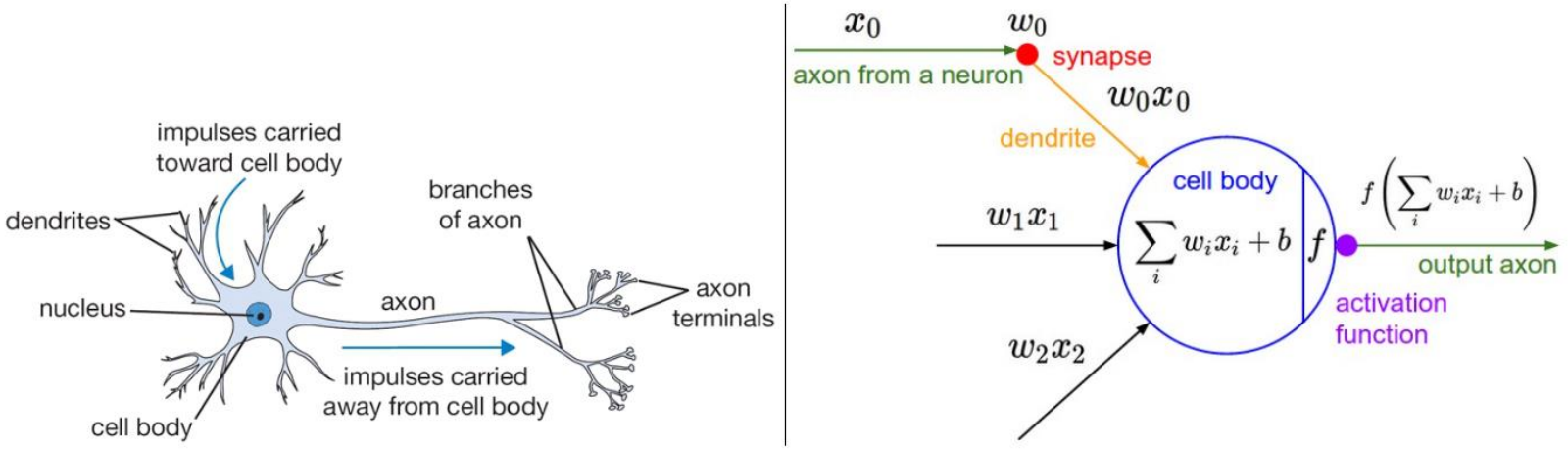
- Pick an initial random weight vector.
- Apply the linear unit to all training examples
- Compute Δw_i for each weight according to weight update rule
- Update each weight w_i by adding Δw_i
- Repeat this process

Limitations of Perceptron

- A perceptron can only draw one straight decision boundary
- This works for simple problems
  - AND
  - OR
  - Linearly separable data
- But it fails when the classes require a nonlinear boundary
- To learn complex patterns, we need hidden layers and nonlinear activation functions

Multilayer Perceptron (MLP)

- An **MLP** is a neural network with one or more hidden layers between the input and output layers
- It is a feedforward network, meaning information moves from input → hidden layer(s) → output
- Compared to a single perceptron, an MLP can learn more complex patterns
- This is possible because MLPs use nonlinear activation functions such as **ReLU**, **sigmoid**, or **tanh**
- The activation function adds nonlinearity, so the network is not limited to a single straight decision boundary
- Because of hidden layers and nonlinear activations, MLPs can solve problems that are not linearly separable, such as XOR



A cartoon drawing of a biological neuron (left) and its mathematical model (right).

FIGURE TAKEN FROM HTTPS://CS231N.GITHUB.IO/NEURAL-NETWORKS-1/

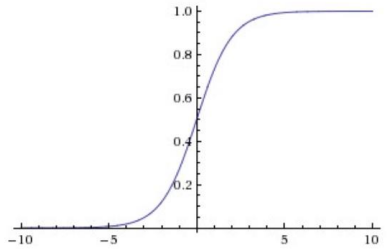
Activation Functions: Sigmoid, Tanh, ReLU

- Activation functions transform the weighted sum of inputs before passing it to the next layer
- They are important because they add nonlinearity to the network
- Without nonlinear activation functions, even a multilayer network would behave like a linear model

Function	Output range	Main idea
Sigmoid	0 to 1	Squashes values like a probability
Tanh	-1 to 1	Similar to sigmoid, but zero-centered
ReLU	0 to ∞	Keeps positive values, turns negative values into 0

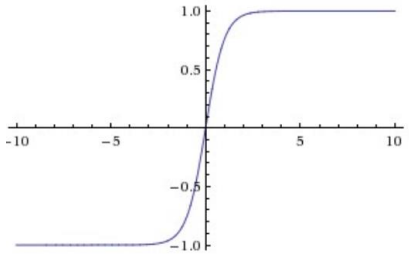
Activation Functions: Choosing an Activation Function

- **Sigmoid**
  - Useful when the output should be between 0 and 1
  - Often used in the output layer for binary classification
  - Can suffer from vanishing gradients
  - Sigmoid outputs are not zero centered
    - The gradient on the weights will become either all positive, or all negative during backpropagation
    - Zig zagging in the weight updates



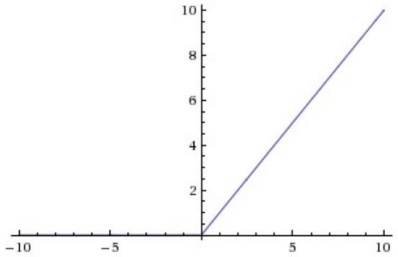
$$f(x) = \frac{1}{1 + e^{-x}}$$

- **Tanh**
  - Outputs values between -1 and 1
  - Better than sigmoid for hidden layers because it is zero centered
  - Can still suffer from vanishing gradients
  - May have exploding gradients problem (all gradients positive)
    - Unstable optimization



$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

- **ReLU**
  - Most common choice for hidden layers
  - Simple and fast
  - Helps reduce the vanishing gradient problem
  - Can suffer from the dying ReLU problem when neurons always output 0



$$f(x) = \max(0, x)$$

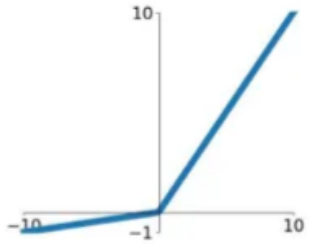
- Use ReLU in hidden layers
- Use sigmoid in the output layer for binary classification

How to mitigate dying ReLU problem?

- **Leaky ReLU**s are one attempt to fix the **dying ReLU** problem.
- Instead of the function being zero when x < 0, a leaky ReLU will instead have a small positive slope (of 0.01, or so).
- Leaky ReLU has the form $f(x) = \max(\alpha x, x)$, where α is a small positive constant.
- Another solution is using Parametric ReLU (**PReLU**).
- Similar to leaky ReLU but allows the slope to be learned during training.

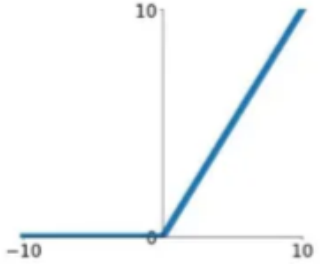
Leaky ReLU

$\max(0.1x, x)$



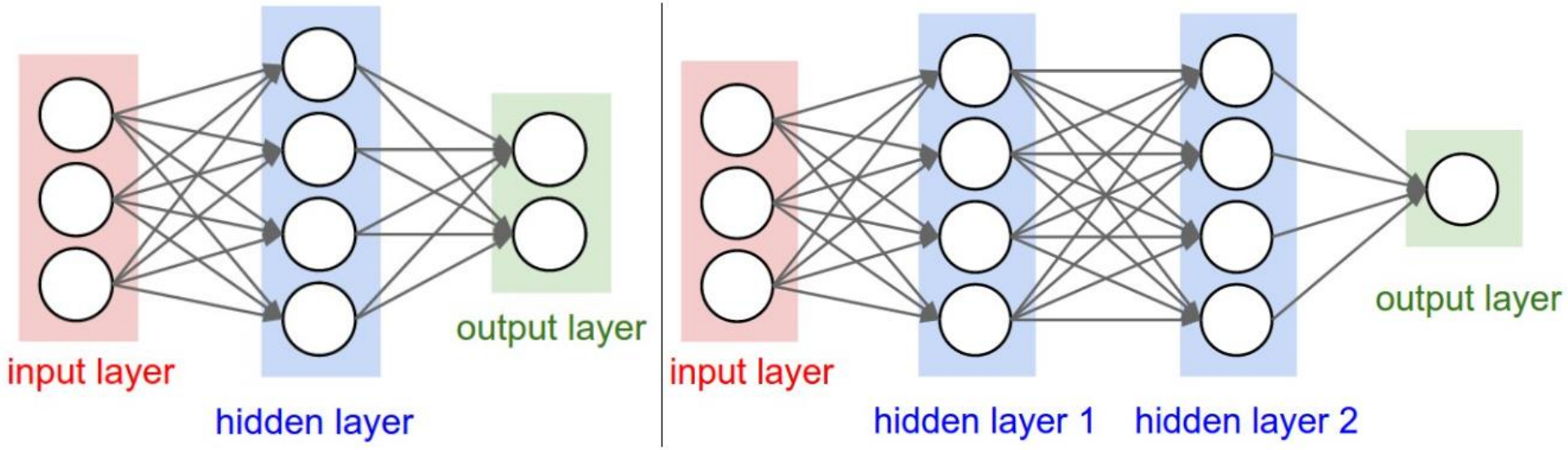
ReLU

$\max(0, x)$



Multilayer ANN / MLP

- Note that the input layer is not counted, because it simply transmits the data and no computation is performed in that layer.

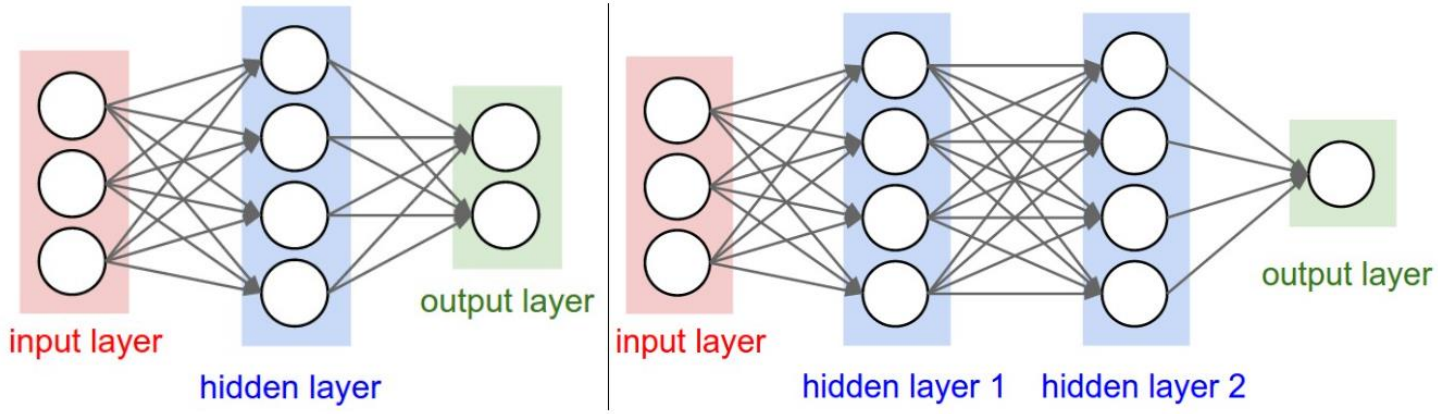


Left: A 2-layer Neural Network (one hidden layer of 4 neurons (or units) and one output layer with 2 neurons), and three inputs.
Right: A 3-layer neural network with three inputs, two hidden layers of 4 neurons each and one output layer. Notice that in both cases there are connections (synapses) between neurons across layers, but not within a layer.

FIGURE TAKEN FROM HTTPS://CS231N.GITHUB.IO/NEURAL-NETWORKS-1/

Multilayer ANN / MLP

- Left network has 4 + 2 = 6 neurons, [3 x 4] + [4 x 2] = 20 weights and 4 + 2 = 6 biases, for a total of 26 learnable parameters.
- Right network has 4 + 4 + 1 = 9 neurons, [3 x 4] + [4 x 4] + [4 x 1] = 12 + 16 + 4 = 32 weights and 4 + 4 + 1 = 9 biases, for a total of 41 learnable parameters.



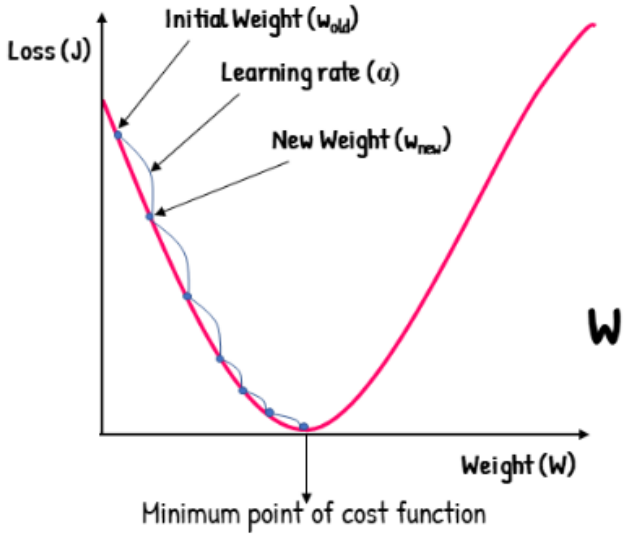
Left: A 2-layer Neural Network (one hidden layer of 4 neurons (or units) and one output layer with 2 neurons), and three inputs.
Right: A 3-layer neural network with three inputs, two hidden layers of 4 neurons each and one output layer. Notice that in both cases there are connections (synapses) between neurons across layers, but not within a layer.

FIGURE TAKEN FROM HTTPS://CS231N.GITHUB.IO/NEURAL-NETWORKS-1/

Training MLP with Backpropagation

- In the single-layer neural network, the training process is easy
- The error can be computed as a direct function of the weights, which allows easy gradient computation.
- In the case of multi-layer networks, the problem is that the loss is a complicated composite function of the weights in earlier layers.
- The gradient of a composite function is computed using the **backpropagation algorithm**.
- Backpropagation algorithm calculates the gradients (partial derivatives) of the cost function with respect to each weight and bias in the network.
- Backpropagation algorithm leverages the chain rule of differential calculus, which computes the error gradients in terms of summations of local-gradient products over the various paths from a node to the output.

Backpropagation



$$w_{new} = w_{old} - \alpha \frac{\delta J}{\delta w}$$

- Two phases
  - Forward phase (forward pass)
  - Backward phase (backward pass)
- **Forward Pass**
  - During forward pass, input data flows through the network layer by layer.
  - Each layer computes an output based on its weights, biases, and activation function.
- **Compute Loss** (After forward pass & Before Backward pass)
  - Compare the predicted output with the actual target (ground truth) to calculate the loss (error).
  - The derivative of this loss now needs to be computed with respect to the weights in all layers in the backwards phase
- **Backward Pass**
  - Start from the output layer and move backward through the layers.
  - Compute the gradient of the loss with respect to each weight and bias using the chain rule.
  - Update the weights and biases using the gradients and the learning rate.

Chain Rule

- We define chain rule as

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

where y is a function of u and u is a function of x. Thus we have the composite function y.

- Example:

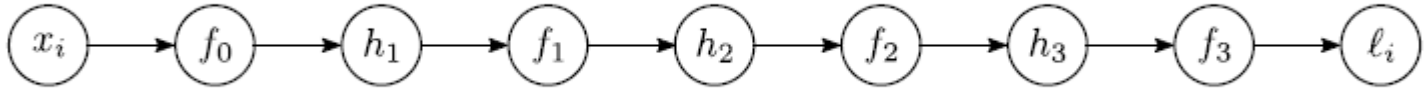
$$y = (5x + 25)^3 \; where \; u = 5x + 25$$

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx} = \frac{d}{du}u^3 \frac{d}{dx}5x + 25 = (3u^2) * 5\frac{d}{dx}x + 5 = 3 * (5x + 25) * 5 = 75(x + 5)$$

Chain Rule

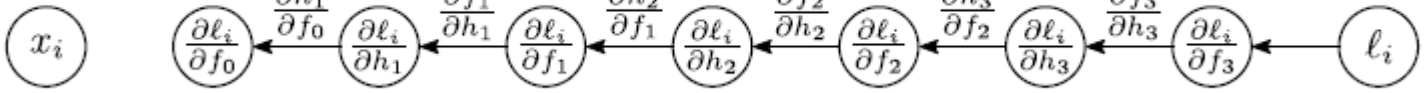
- Forward Pass:

$$
\begin{aligned}
f_0 &= \beta_0 + \omega_0 \cdot x_i \\
h_1 &= \sin[f_0] \\
f_1 &= \beta_1 + \omega_1 \cdot h_1 \\
h_2 &= \exp[f_1] \\
f_2 &= \beta_2 + \omega_2 \cdot h_2 \\
h_3 &= \cos[f_2] \\
f_3 &= \beta_3 + \omega_3 \cdot h_3 \\
\ell_i &= (f_3 - y_i)^2.
\end{aligned}
$$



- Backward Pass:

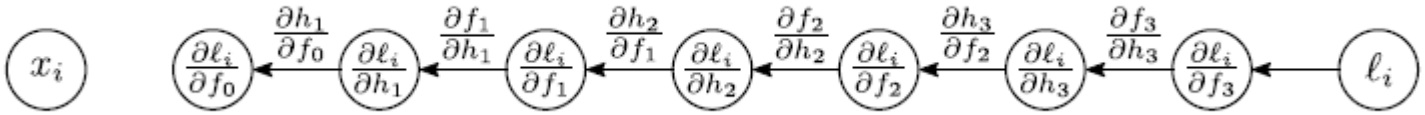
$$\frac{\partial \ell_i}{\partial f_3} = 2(f_3 - y_i).$$



Chain Rule

- Backward Pass:

$$\frac{\partial \ell_i}{\partial f_3} = 2(f_3 - y_i).$$



- Using chain rule

$$\frac{\partial \ell_i}{\partial h_3} = \frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3}.$$

$$\begin{aligned}
\frac{\partial \ell_i}{\partial f_2} &= \frac{\partial h_3}{\partial f_2}\left(\frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3}\right) \\
\frac{\partial \ell_i}{\partial h_2} &= \frac{\partial f_2}{\partial h_2}\left(\frac{\partial h_3}{\partial f_2}\frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3}\right) \\
\frac{\partial \ell_i}{\partial f_1} &= \frac{\partial h_2}{\partial f_1}\left(\frac{\partial f_2}{\partial h_2}\frac{\partial h_3}{\partial f_2}\frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3}\right) \\
\frac{\partial \ell_i}{\partial h_1} &= \frac{\partial f_1}{\partial h_1}\left(\frac{\partial h_2}{\partial f_1}\frac{\partial f_2}{\partial h_2}\frac{\partial h_3}{\partial f_2}\frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3}\right) \\
\frac{\partial \ell_i}{\partial f_0} &= \frac{\partial h_1}{\partial f_0}\left(\frac{\partial f_1}{\partial h_1}\frac{\partial h_2}{\partial f_1}\frac{\partial f_2}{\partial h_2}\frac{\partial h_3}{\partial f_2}\frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3}\right).
\end{aligned}$$